

Comandos Básicos en Python

Uso de `input()`, `print()`, variables enteras y el comando `int()`

Departamento de Informática

6 de agosto de 2026

Índice

1. Introducción	2
2. El comando <code>print()</code>	2
2.1. Sintaxis básica	2
2.2. Salida con múltiples elementos	2
2.3. Formateo de cadenas	2
3. El comando <code>input()</code>	2
3.1. Sintaxis básica	3
3.2. Importante: <code>input()</code> siempre devuelve una cadena de texto	3
4. Variables numéricas enteras y operaciones	3
4.1. Declaración de variables enteras	3
4.2. Operadores aritméticos básicos	3
4.3. Ejemplos de operaciones	3
5. El comando <code>int()</code>	4
5.1. Convertir una cadena a entero	4
5.2. Convertir un número decimal a entero (truncamiento)	4
5.3. Uso combinado con <code>input()</code>	4
5.4. Manejo de errores	4
6. Ejemplos completos	4
6.1. Ejemplo 1: Calculadora básica	4
6.2. Ejemplo 2: Conversión de unidades	5
6.3. Ejemplo 3: Cálculo de promedio	5
7. Resumen y buenas prácticas	5
8. Ejercicios propuestos	5

1. Introducción

En este documento exploraremos cuatro conceptos fundamentales en la programación con Python:

- El comando `print()`, para mostrar información en pantalla.
- El comando `input()`, para recibir datos del usuario.
- Las variables numéricas de tipo entero (`int`) y las operaciones aritméticas básicas.
- El comando `int()`, para convertir datos a números enteros.

Estos elementos son la base para crear programas interactivos y realizar cálculos.

2. El comando `print()`

El comando `print()` se utiliza para mostrar mensajes, valores de variables o resultados de operaciones en la consola.

2.1. Sintaxis básica

```
1 print("Hola, mundo")
2 print(42)
3 print(3 + 5)
```

2.2. Salida con múltiples elementos

Podemos mostrar varios elementos separándolos por comas:

```
1 nombre = "Ana"
2 edad = 25
3 print("Nombre:", nombre, "Edad:", edad)
```

2.3. Formateo de cadenas

Existen varias formas de formatear la salida:

```
1 # Usando f-strings (recomendado)
2 print(f"El resultado es {resultado}")
3
4 # Usando el m todo format()
5 print("El resultado es {}".format(resultado))
6
7 # Usando el operador %
8 print("El resultado es %d" % resultado)
```

3. El comando `input()`

El comando `input()` permite que el programa reciba datos escritos por el usuario durante la ejecución.

3.1. Sintaxis básica

```
1 nombre = input(" Cmo te llamas? ")
2 print("Hola", nombre)
```

3.2. Importante: input() siempre devuelve una cadena de texto

Todos los datos ingresados con `input()` son de tipo `str` (cadena de caracteres), incluso si el usuario escribe números.

```
1 edad = input("Ingresa tu edad: ")
2 print(type(edad)) # Muestra: <class 'str'>
```

4. Variables numéricas enteras y operaciones

Las variables de tipo entero (`int`) almacenan números sin parte decimal. Con ellas podemos realizar operaciones aritméticas básicas.

4.1. Declaración de variables enteras

```
1 a = 10
2 b = 3
3 c = -5
```

4.2. Operadores aritméticos básicos

Operador	Descripción	Ejemplo
+	Suma	$10 + 3 = 13$
-	Resta	$10 - 3 = 7$
*	Multiplicación	$10 * 3 = 30$
/	División (real)	$10 / 3 = 3.333...$
//	División entera	$10 // 3 = 3$
%	Módulo (resto)	$10 \% 3 = 1$
**	Potencia	$10 ** 3 = 1000$

Cuadro 1: Operadores aritméticos en Python

4.3. Ejemplos de operaciones

```
1 x = 15
2 y = 4
3
4 suma = x + y
5 resta = x - y
6 multiplicacion = x * y
7 division_real = x / y
8 division_entera = x // y
9 resto = x % y
```

```

10 potencia = x ** y
11
12 print(f"Suma: {suma}")
13 print(f"Resta: {resta}")
14 print(f"Multiplicaci n: {multiplicacion}")
15 print(f"Divisi n real: {division_real}")
16 print(f"Divisi n entera: {division_entera}")
17 print(f"Resto: {resto}")
18 print(f"Potencia: {potencia}")

```

5. El comando int()

El comando int() se utiliza para convertir otros tipos de datos a números enteros.

5.1. Convertir una cadena a entero

```

1 numero_texto = "25"
2 numero_entero = int(numero_texto)
3 print(numero_entero + 5) # Resultado: 30

```

5.2. Convertir un número decimal a entero (truncamiento)

decimal = 7.89 entero = int(decimal) Resultado: 7 print(entero)

5.3. Uso combinado con input()

La combinación más común es:

```

1 edad = int(input("Ingresa tu edad: "))
2 anio_nacimiento = 2026 - edad
3 print(f"Naciste aproximadamente en el a o {anio_nacimiento}")

```

5.4. Manejo de errores

Es importante considerar que int() generará un error si la cadena no representa un número válido.

```

1 # Esto causar un error:
2 # numero = int("hola")
3
4 # Podemos manejar el error con try/except
5 try:
6     numero = int(input("Ingresa un n mero: "))
7     print(f"El n mero es {numero}")
8 except ValueError:
9     print("Error: Debes ingresar un n mero entero v lido.")

```

6. Ejemplos completos

6.1. Ejemplo 1: Calculadora básica

```

1 print("=== CALCULADORA BASICA ===")
2 num1 = int(input("Ingresa el primer numero: "))
3 num2 = int(input("Ingresa el segundo numero: "))
4
5 print(f"\n{num1} + {num2} = {num1 + num2}")
6 print(f"{num1} - {num2} = {num1 - num2}")
7 print(f"{num1} * {num2} = {num1 * num2}")
8 print(f"{num1} / {num2} = {num1 / num2}")
9 print(f"{num1} // {num2} = {num1 // num2}")
10 print(f"{num1} % {num2} = {num1 % num2}")
11 print(f"{num1} ** {num2} = {num1 ** num2}")

```

6.2. Ejemplo 2: Conversión de unidades

```

1 print("=== CONVERSION DE GRADOS ===")
2 celsius = int(input("Ingresa la temperatura en C : "))
3 fahrenheit = (celsius * 9/5) + 32
4 print(f"{celsius} C equivalen a {fahrenheit} F ")

```

6.3. Ejemplo 3: Cálculo de promedio

```

1 print("=== CALCULO DE PROMEDIO ===")
2 nota1 = int(input("Ingresa la primera nota: "))
3 nota2 = int(input("Ingresa la segunda nota: "))
4 nota3 = int(input("Ingresa la tercera nota: "))
5
6 promedio = (nota1 + nota2 + nota3) / 3
7 print(f"El promedio es: {promedio:.2f}") # :.2f muestra 2 decimales

```

7. Resumen y buenas prácticas

- **print():** Úsalo para mostrar información al usuario.
- **input():** Úsalo para recibir datos; siempre devuelve texto.
- **int():** Convierte texto o números decimales a enteros.
- **Operaciones:** Recuerda que la división `/` siempre devuelve un número decimal (float).
- **Seguridad:** Siempre valida la entrada del usuario para evitar errores.
- **Lectura:** Nombra tus variables de forma descriptiva (ej. `edad`, `precio`).

8. Ejercicios propuestos

1. Crea un programa que pida dos números enteros y muestre su suma, resta, multiplicación y división entera.
2. Escribe un programa que calcule el área de un rectángulo a partir de su base y altura (enteros).

3. Desarrolla un programa que pida un número de tres dígitos y muestre sus dígitos por separado (usando operaciones con enteros).
4. Crea un conversor de segundos a horas, minutos y segundos (usando división entera y módulo).

Usos de `print()` y `print(". ")` en Python

Documento Técnico

August 6, 2026

Abstract

Este documento explora los diversos usos de la función `print()` en Python, con un enfoque especial en la variante `print(". ")`. Se analizan desde los aspectos más básicos hasta aplicaciones avanzadas, incluyendo ejemplos prácticos que ilustran su utilidad en la depuración, visualización de datos y control de flujo.

1 Introducción

La función `print()` es una de las herramientas más fundamentales en Python. Permite mostrar información en la consola, lo que es esencial para la depuración, la interacción con el usuario y la presentación de resultados. Aunque parece simple, `print()` ofrece una gran flexibilidad a través de sus parámetros y su comportamiento.

En particular, `print(". ")` puede parecer trivial, pero tiene aplicaciones interesantes en la creación de barras de progreso, separadores visuales y patrones de texto.

2 Usos básicos de `print()`

2.1 Imprimir cadenas y variables

El uso más común es imprimir texto o el valor de variables:

```
1 nombre = "Ana"
2 print("Hola, ", nombre)
```

Esto produce: Hola, Ana

2.2 Imprimir múltiples elementos

Se pueden pasar varios argumentos separados por comas:

```
1 print("El resultado es", 42, "y el error es", 0.5)
```

2.3 El parámetro `sep`

Permite definir el separador entre los elementos:

```
1 print("uno", "dos", "tres", sep="-")
```

Salida: uno-dos-tres

2.4 El parámetro end

Controla lo que se imprime al final (por defecto es salto de línea):

```
1 print("Hola", end=" ")
2 print("Mundo")
```

Salida: Hola Mundo

3 Usos de print(". ")

Imprimir un punto puede parecer sencillo, pero tiene varios usos prácticos.

3.1 Separador visual

Se usa para crear líneas divisorias o marcas en la salida:

```
1 print(".")
2 print("-----")
3 print(".")
```

3.2 Indicador de progreso

En bucles largos, `print(".", end="")` imprime puntos en la misma línea para mostrar avance:

```
1 import time
2 for i in range(10):
3     time.sleep(0.5)
4     print(".", end="", flush=True)
5 print("    Completado    !")
```

3.3 Formateo de salida

Ayuda a crear patrones visuales como menús o tablas simples:

```
1 print("-----")
2 print("| Opci n 1 | Opci n 2 |")
3 print("-----")
```

3.4 Depuración visual

Se puede usar para marcar puntos específicos en la ejecución:

```
1 print(".")
2 x = 5 * 3
3 print(".")
4 print("Resultado:", x)
```

4 Uso avanzado: barras de progreso y efectos visuales

Combinando `print(".", end="")` con bucles y `flush=True`, se pueden crear barras de progreso simples:

```
1 import time
2 for i in range(1, 11):
3     time.sleep(0.3)
4     print(".", end="\r", flush=True)
5 print(" Proceso completado.")
```

También se puede usar para animar texto o crear menús interactivos en consola.

5 Conclusiones

La función `print()` en Python es mucho más versátil de lo que aparenta. Si bien `print(".")` parece insignificante, tiene aplicaciones útiles en visualización, depuración y retroalimentación al usuario. Dominar estos detalles permite escribir código más claro, interactivo y profesional.

References

- [1] Python Software Foundation. *Python Documentation: Built-in Functions*.
<https://docs.python.org/3/library/functions.html#print>

Uso de la Biblioteca `math` en Python

Documentación Técnica

23 de agosto de 2026

Resumen

Este documento ofrece una guía completa sobre el uso de la biblioteca estándar `math` de Python. Se describen las funciones más importantes, constantes matemáticas y ejemplos prácticos de aplicación en distintos contextos.

1. Introducción

La biblioteca `math` de Python proporciona acceso a funciones matemáticas definidas por el estándar C. Es parte de la biblioteca estándar, por lo que no requiere instalación adicional. Está diseñada para trabajar con números de punto flotante y ofrece herramientas útiles para cálculos avanzados.

2. Importar la biblioteca

Para usar la biblioteca `math`, se debe importar. Se recomienda la siguiente sintaxis:

```
import math
```

También es posible importar funciones específicas:

```
from math import sqrt, pi
```

3. Constantes matemáticas

La biblioteca incluye constantes fundamentales:

- `math.pi`: $\pi \approx 3,141592653589793$
- `math.e`: $e \approx 2,718281828459045$
- `math.tau`: $\tau = 2\pi \approx 6,283185307179586$
- `math.inf`: Infinito positivo
- `math.nan`: No es un número (Not a Number)

4. Funciones principales

4.1. Funciones trigonométricas

Todas las funciones trabajan en radianes.

```
math.sin(x)    # Seno
math.cos(x)    # Coseno
math.tan(x)    # Tangente
math.asin(x)   # Arcoseno
math.acos(x)   # Arcocoseno
math.atan(x)   # Arcotangente
math.atan2(y, x) # Arcotangente de y/x
```

4.2. Funciones hiperbólicas

```
math.sinh(x)
math.cosh(x)
math.tanh(x)
math.asinh(x)
math.acosh(x)
math.atanh(x)
```

4.3. Funciones exponenciales y logarítmicas

```
math.exp(x)      # e^x
math.log(x)       # Logaritmo natural (base e)
math.log(x, base) # Logaritmo en base especificada
math.log10(x)     # Logaritmo base 10
math.log2(x)      # Logaritmo base 2
math.pow(x, y)    # x^y (similar a x**y)
math.sqrt(x)      # Raíz cuadrada
```

4.4. Funciones de redondeo y valor absoluto

```
math.ceil(x)      # Redondeo hacia arriba
math.floor(x)     # Redondeo hacia abajo
math.trunc(x)     # Truncamiento (parte entera)
math.fabs(x)      # Valor absoluto
math.modf(x)      # Devuelve parte fraccionaria y entera
```

4.5. Funciones combinatorias

```
math.factorial(n) # Factorial de n
math.comb(n, k)   # Combinaciones de n en k
math.perm(n, k)   # Permutaciones de n en k
```

4.6. Otras funciones útiles

```
math.gcd(a, b)      # Máximo común divisor
math.lcm(a, b)      # Mínimo común múltiplo (Python 3.9+)
math.isqrt(n)       # Raíz cuadrada entera
math.dist(p, q)     # Distancia euclidiana entre dos puntos
math.hypot(x, y)    # Hipotenusa  $\sqrt{x^2 + y^2}$ 
```

5. Ejemplos prácticos

5.1. Cálculo del área de un círculo

```
import math

radio = 5.0
area = math.pi * math.pow(radio, 2)
print(f"Área del círculo: {area:.2f}")
```

5.2. Solución de ecuación cuadrática

```
import math

a, b, c = 1, -3, 2
discriminante = math.pow(b, 2) - 4*a*c

if discriminante >= 0:
    raiz1 = (-b + math.sqrt(discriminante)) / (2*a)
    raiz2 = (-b - math.sqrt(discriminante)) / (2*a)
    print(f"Raíces: {raiz1}, {raiz2}")
else:
    print("Raíces complejas")
```

5.3. Conversión de grados a radianes

```
import math

grados = 45
radianes = math.radians(grados)
seno = math.sin(radianes)
print(f"Seno de {grados}°: {seno:.4f}")
```

5.4. Número de combinaciones posibles

```
import math

n = 10
k = 3
combinaciones = math.comb(n, k)
print(f"Combinaciones de {n} en {k}: {combinaciones}")
```

6. Errores comunes y buenas prácticas

- **Dominio:** Algunas funciones como `math.log` o `math.sqrt` solo aceptan números positivos. Intentar con valores negativos lanza un `ValueError`.
- **Precisión:** Los resultados son de punto flotante; comparar directamente con números exactos puede dar resultados inesperados.
- **No usar `math.pow` para exponentes enteros:** Preferir el operador `**` que es más eficiente.
- **Importar solo lo necesario:** Para mejorar el rendimiento y legibilidad, importar funciones específicas cuando sea posible.

7. Comparativa: `math` vs `numpy`

Función	<code>math</code>	<code>numpy</code>
Soporte para arrays	No	Sí
Rendimiento en escalares	Rápido	Similar
Funciones adicionales	Básicas	Avanzadas (FFT, álgebra lineal)
Peso de la biblioteca	Ligera	Pesada

Cuadro 1: Diferencias entre `math` y `numpy`

8. Conclusión

La biblioteca `math` es una herramienta fundamental para cualquier programador de Python que necesite realizar operaciones matemáticas. Su simplicidad, eficiencia y amplia cobertura de funciones la convierten en la opción ideal para cálculos cotidianos sin la necesidad de bibliotecas externas pesadas.

Guía completa para definir funciones en Python

Documento generado automáticamente

23 de agosto de 2026

Resumen

Este documento proporciona una guía detallada sobre el uso del comando `def` en Python para definir funciones. Se abordan los conceptos fundamentales, la sintaxis, los parámetros, el valor de retorno, el alcance de las variables y buenas prácticas. El objetivo es ofrecer un recurso claro y práctico para programadores de todos los niveles.

Índice

1. Introducción	2
2. Sintaxis básica de <code>def</code>	2
2.1. Componentes clave	2
2.2. Ejemplo básico	3
3. Parámetros y argumentos	3
3.1. Parámetros posicionales	3
3.2. Parámetros con valores por defecto	3
3.3. Parámetros arbitrarios: <code>*args</code> y <code>**kwargs</code>	3
3.4. Parámetros solo con nombre (Keyword-only)	4
4. El valor de retorno: <code>return</code>	4
5. Alcance de las variables (Scope)	5
5.1. Funciones anidadas y <code>nonlocal</code>	5
6. Documentación de funciones: Docstrings	6
7. Buenas prácticas al definir funciones	6
8. Ejemplos completos	6
8.1. Función que procesa una lista	6
8.2. Función con validación de entrada	7
9. Funciones anónimas: <code>lambda</code>	7
10. Conclusión	7

1. Introducción

En Python, las funciones son bloques de código reutilizables que realizan una tarea específica. La palabra clave **def** es la que nos permite *definir* una función. Las funciones son esenciales para:

- **Organizar** el código en partes lógicas.
- **Reutilizar** código sin duplicarlo.
- **Abstraer** la complejidad.
- **Facilitar** el mantenimiento y la depuración.

2. Sintaxis básica de def

La sintaxis general para definir una función es la siguiente:

```
1 def nombre_funcion(parametro1, parametro2, ...):  
2     """Docstring opcional: explica que hace la funcion."""  
3     # Cuerpo de la funcion  
4     # ...  
5     return valor_de_retorno # Opcional
```

Listing 1: Sintaxis básica de una función en Python

2.1. Componentes clave

1. **Palabra clave def**: Indica el inicio de la definición.
2. **Nombre de la función**: Debe ser descriptivo y seguir las convenciones de nombres (snake_case).
3. **Paréntesis ()**: Encierran los parámetros que la función puede recibir. Pueden estar vacíos.
4. **Dos puntos ::**: Indican el final de la cabecera y el inicio del bloque de código.
5. **Indentación**: El cuerpo de la función debe estar indentado (generalmente 4 espacios).
6. **Comentario/docstring**: Una cadena de texto que describe el propósito de la función.
7. **Declaración return**: Devuelve un valor (opcional). Si no se usa, la función devuelve **None**.

2.2. Ejemplo básico

```
1 def saludar(nombre):
2     """Esta funcion saluda a la persona pasada como argumento."""
3     mensaje = f" Hola , {nombre}!"
4     return mensaje
5
6 # Llamada a la funcion
7 resultado = saludar("Ana")
8 print(resultado) # Salida:  Hola , Ana!
```

Listing 2: Ejemplo de una función simple

3. Parámetros y argumentos

Python ofrece una gran flexibilidad en la definición de parámetros. A continuación se describen los distintos tipos.

3.1. Parámetros posicionales

Son los más comunes. El orden en el que se pasan los argumentos debe coincidir con el de los parámetros.

```
1 def resta(a, b):
2     return a - b
3
4 print(resta(10, 3)) # Salida: 7
5 # El orden importa
6 print(resta(3, 10)) # Salida: -7
```

Listing 3: Parámetros posicionales

3.2. Parámetros con valores por defecto

Se asignan valores por defecto a los parámetros. Si la función es llamada sin pasar argumentos para ellos, se usan los valores predeterminados.

```
1 def presentar(nombre, edad=25, pais="Desconocido"):
2     return f"{nombre} tiene {edad} a os y vive en {pais}."
3
4 print(presentar("Carlos")) # Usa edad y pais por defecto
5 print(presentar("Maria", 30)) # Usa pais por defecto
6 print(presentar("Luis", 28, "Chile")) # Todos personalizados
```

Listing 4: Parámetros con valor por defecto

Importante: Los parámetros por defecto se evalúan una sola vez en el momento de la definición. No se recomienda usar tipos mutables (listas, diccionarios) como valores por defecto a menos que se tenga cuidado.

3.3. Parámetros arbitrarios: *args y **kwargs

Permiten pasar un número variable de argumentos.

- ***args**: Recibe una tupla de argumentos posicionales variables.
- ****kwargs**: Recibe un diccionario de argumentos con nombre (keyword arguments).

```

1 def funcion_variable(*args, **kwargs):
2     print("Argumentos posicionales:", args)
3     print("Argumentos con nombre:", kwargs)
4
5 funcion_variable(1, 2, 3, nombre="Pedro", edad=30)
6 # Salida:
7 # Argumentos posicionales: (1, 2, 3)
8 # Argumentos con nombre: {'nombre': 'Pedro', 'edad': 30}

```

Listing 5: Uso de *args y **kwargs

3.4. Parámetros solo con nombre (Keyword-only)

Se pueden forzar a que ciertos parámetros solo se pasen usando su nombre (después de *).

```

1 def configurar(usuario, *, clave, puerto=80):
2     return f"Usuario: {usuario}, Clave: {clave}, Puerto: {puerto}"
3
4 # Llamada correcta
5 print(configurar("admin", clave="1234")) # OK
6 # Llamada incorrecta (esto daría error)
7 # configurar("admin", "1234") # TypeError

```

Listing 6: Parámetros solo con nombre

4. El valor de retorno: return

La declaración **return** finaliza la ejecución de la función y devuelve un valor. Una función puede:

- Devolver un solo valor.
- Devolver múltiples valores (como una tupla).
- No tener **return** (en ese caso devuelve **None**).
- Tener múltiples puntos de retorno.

```

1 # Retorno multiple
2 def operaciones(x, y):
3     suma = x + y
4     resta = x - y
5     producto = x * y
6     return suma, resta, producto # Devuelve una tupla
7
8 s, r, p = operaciones(10, 5)
9 print(f"Suma: {s}, Resta: {r}, Producto: {p}")
10

```

```

11 # Funcion sin return explicito
12 def vacio():
13     pass
14
15 print(vacio()) # Salida: None

```

Listing 7: Diferentes formas de usar return

5. Alcance de las variables (Scope)

Las variables definidas dentro de una función tienen alcance *local*. Las variables definidas fuera tienen alcance *global*. Para modificar una variable global desde dentro de una función, se usa la palabra clave `global`.

```

1 variable_global = 10
2
3 def modificar_global():
4     global variable_global
5     variable_global = 20
6
7 def usar_local():
8     variable_local = 5
9     return variable_local
10
11 print(variable_global) # 10
12 modificar_global()
13 print(variable_global) # 20
14 print(usar_local())    # 5
15 # print(variable_local) # Esto daria error (no definida fuera)

```

Listing 8: Alcance local y global

5.1. Funciones anidadas y nonlocal

Python permite definir funciones dentro de otras funciones. Para modificar una variable de la función envolvente, se usa `nonlocal`.

```

1 def exterior():
2     valor = 100
3     def interior():
4         nonlocal valor
5         valor += 50
6         print(f"Interior: {valor}")
7     interior()
8     print(f"Exterior: {valor}")
9
10 exterior()
11 # Salida:
12 # Interior: 150
13 # Exterior: 150

```

Listing 9: Uso de nonlocal

6. Documentación de funciones: Docstrings

Es una buena práctica documentar las funciones con *docstrings*. Se escriben como la primera línea del cuerpo de la función, entre comillas triples. Herramientas como `help()` o Sphinx los utilizan.

```
1 def calcular_area_circulo(radio):
2     """
3     Calcula el area de un circulo dado su radio.
4
5     Parametros:
6     radio (float): El radio del circulo.
7
8     Retorna:
9     float: El area del circulo.
10    """
11    import math
12    return math.pi * radio ** 2
13
14 help(calcular_area_circulo)
```

Listing 10: Ejemplo de docstring

7. Buenas prácticas al definir funciones

1. **Nombres descriptivos:** Usa verbos que indiquen acción (ej. `calcular_promedio`, `obtener_usuario`).
2. **Una sola responsabilidad:** Cada función debe hacer una cosa y hacerla bien.
3. **Documentar siempre:** Escribe docstrings incluso para funciones simples.
4. **Evitar el uso excesivo de variables globales:** Prefiere pasar valores como parámetros.
5. **Usar tipado estático (Type Hints):** Mejora la legibilidad y ayuda con herramientas de análisis.

```
1 def sumar(a: int, b: int) -> int:
2     """Devuelve la suma de dos numeros enteros."""
3     return a + b
```

Listing 11: Ejemplo con Type Hints

8. Ejemplos completos

8.1. Función que procesa una lista

```
1 def filtrar_pares(lista_numeros):
2     """Filtra los numeros pares de una lista."""
3     return [num for num in lista_numeros if num % 2 == 0]
4
```

```

5 numeros = [1, 2, 3, 4, 5, 6, 7, 8]
6 print(filtrar_pares(numeros)) # [2, 4, 6, 8]

```

Listing 12: Procesamiento de lista

8.2. Función con validación de entrada

```

1 def dividir_seguro(a, b):
2     """Divide dos numeros, manejando la division por cero."""
3     if b == 0:
4         return "Error: Division por cero."
5     return a / b
6
7 print(dividir_seguro(10, 2)) # 5.0
8 print(dividir_seguro(10, 0)) # Error: Division por cero.

```

Listing 13: Validación con manejo de errores

9. Funciones anónimas: lambda

Además de `def`, Python permite definir funciones pequeñas y anónimas con la palabra clave `lambda`. Son útiles para operaciones simples.

```

1 # Sintaxis: lambda argumentos: expresion
2 cuadrado = lambda x: x ** 2
3 print(cuadrado(5)) # 25
4
5 # Uso comun con sorted, filter, map
6 puntos = [(1, 2), (3, 1), (5, -1), (4, 4)]
7 puntos_ordenados = sorted(puntos, key=lambda punto: punto[1])
8 print(puntos_ordenados) # [(5, -1), (3, 1), (1, 2), (4, 4)]

```

Listing 14: Ejemplo de lambda

10. Conclusión

El comando `def` es fundamental en Python y proporciona una manera poderosa y flexible de estructurar el código. Entender cómo usar correctamente los parámetros, el retorno, el alcance de las variables y las buenas prácticas permite escribir programas más limpios, eficientes y fáciles de mantener.

Referencias

- Documentación oficial de Python: <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>
- PEP 8 – Style Guide for Python Code.
- PEP 257 – Docstring Conventions.

Uso de Arreglos en Python

Comandos y Ejemplos Prácticos

Documento Elaborado

August 23, 2026

Abstract

Este documento presenta una guía sobre el uso de arreglos en el lenguaje de programación Python. Se describen los comandos más utilizados para manejar listas (arrays dinámicos) y se incluyen dos ejemplos detallados que ilustran su aplicación en situaciones cotidianas.

1 Introducción

En Python, los arreglos se implementan principalmente mediante **listas**, que son estructuras de datos dinámicas y flexibles. A diferencia de los arreglos tradicionales de otros lenguajes, las listas en Python pueden contener elementos de diferentes tipos y crecer o reducir su tamaño dinámicamente.

2 Comandos Básicos para Arreglos en Python

2.1 Creación de Arreglos

```
1 # Lista vacia
2 mi_lista = []
3
4 # Lista con elementos
5 numeros = [1, 2, 3, 4, 5]
6 mixto = [1, "hola", 3.14, True]
```

2.2 Acceso a Elementos

```
1 # Acceso por indice (comienza en 0)
2 primer = numeros[0]      # 1
3 ultimo = numeros[-1]     # 5
4
5 # Slicing (rebanado)
6 sub_lista = numeros[1:4]  # [2, 3, 4]
```

2.3 Comandos de Modificación

```
1 # Agregar elementos
2 numeros.append(6)      # [1,2,3,4,5,6]
3 numeros.insert(2, 10)  # [1,2,10,3,4,5,6]
4
5 # Eliminar elementos
6 numeros.remove(10)     # [1,2,3,4,5,6]
7 ultimo = numeros.pop() # elimina y devuelve el ultimo
8
9 # Extender lista
10 numeros.extend([7,8,9]) # [1,2,3,4,5,6,7,8,9]
```

2.4 Comandos de Ordenación

```
1 # Ordenar (modifica la lista original)
2 numeros.sort()          # ascendente
3 numeros.sort(reverse=True) # descendente
4
5 # Ordenar sin modificar original
6 nueva = sorted(numeros)
```

2.5 Comandos de Búsqueda

```
1 # Buscar indice
2 indice = numeros.index(5) # devuelve la posicion
3
4 # Contar ocurrencias
5 count = numeros.count(2)
6
7 # Verificar existencia
8 existe = 5 in numeros    # True/False
```

3 Ejemplo 1: Análisis de Calificaciones

Este ejemplo muestra cómo usar arreglos para gestionar y analizar calificaciones de estudiantes.

3.1 Código

```
1 # Ejemplo 1: Sistema de calificaciones
2 calificaciones = [85, 92, 78, 90, 88, 76, 95, 89, 72, 84]
3
4 # Comandos de arreglos utilizados
5 total = sum(calificaciones)
6 promedio = total / len(calificaciones)
7 maxima = max(calificaciones)
8 minima = min(calificaciones)
9
10 # Ordenar para analisis
11 calificaciones.sort()
12 print(f"Calificaciones ordenadas: {calificaciones}")
```

```

13
14 # Estadísticas
15 print(f"Promedio: {promedio:.2f}")
16 print(f"Calificación mas alta: {maxima}")
17 print(f"Calificación mas baja: {minima}")
18
19 # Filtrar aprobados (>70)
20 aprobados = [nota for nota in calificaciones if nota >= 70]
21 print(f"Aprobados: {len(aprobados)}")
22 print(f"Desaprobados: {len(calificaciones) - len(aprobados)}")
23
24 # Agregar nueva calificación
25 nueva_nota = 91
26 calificaciones.append(nueva_nota)
27 print(f"Nuevo promedio: {sum(calificaciones)/len(calificaciones):.2f}")

```

3.2 Resultados

Calificaciones ordenadas: [72, 76, 78, 84, 85, 88, 89, 90, 92, 95]
Promedio: 84.90
Calificación mas alta: 95
Calificación mas baja: 72
Aprobados: 10
Desaprobados: 0
Nuevo promedio: 85.45

4 Ejemplo 2: Gestión de Inventario

Este ejemplo demuestra el uso de arreglos para manejar un inventario de productos.

4.1 Código

```

1 # Ejemplo 2: Sistema de inventario
2 productos = ["Laptop", "Mouse", "Teclado", "Monitor", "USB"]
3 precios = [1200, 25, 45, 300, 15]
4 cantidad = [5, 20, 15, 8, 50]
5
6 # Mostrar inventario
7 print("Inventario actual:")
8 for i in range(len(productos)):
9     print(f"{productos[i]}: {cantidad[i]} unidades a ${precios[i]}")
10
11 # Calcular valor total del inventario
12 valor_total = sum(precios[i] * cantidad[i] for i in range(len(productos)
13 ))
14 print(f"\nValor total del inventario: ${valor_total}")
15
16 # Agregar nuevo producto
17 productos.append("Tablet")
18 precios.append(450)
19 cantidad.append(10)
20
21 # Actualizar cantidad de un producto existente

```

```

21 indice = productos.index("Mouse")
22 cantidad[indice] = 25
23
24 # Eliminar producto sin stock
25 if 0 in cantidad:
26     indice_cero = cantidad.index(0)
27     productos.pop(indice_cero)
28     precios.pop(indice_cero)
29     cantidad.pop(indice_cero)
30
31 # Mostrar productos con bajo stock (< 10)
32 print("\nProductos con bajo stock:")
33 bajo_stock = [productos[i] for i in range(len(cantidad)) if cantidad[i]
34               < 10]
35
36 # Ordenar productos por precio
37 productos_ordenados = sorted(zip(productos, precios, cantidad),
38                               key=lambda x: x[1])
39 print("\nProductos ordenados por precio (menor a mayor):")
40 for prod, prec, cant in productos_ordenados:
41     print(f"{prod}: ${prec} - {cant} unidades")

```

4.2 Resultados

Inventario actual:

Laptop: 5 unidades a \$1200

Mouse: 20 unidades a \$25

Teclado: 15 unidades a \$45

Monitor: 8 unidades a \$300

USB: 50 unidades a \$15

Valor total del inventario: \$10075

Productos con bajo stock:

['Laptop', 'Monitor']

Productos ordenados por precio (menor a mayor):

USB: \$15 - 50 unidades

Mouse: \$25 - 25 unidades

Teclado: \$45 - 15 unidades

Monitor: \$300 - 8 unidades

Tablet: \$450 - 10 unidades

Laptop: \$1200 - 5 unidades

5 Conclusión

Los arreglos (listas) en Python son herramientas fundamentales que permiten almacenar y manipular colecciones de datos de manera eficiente. Los comandos presentados (`append()`, `pop()`, `sort()`, `index()`, etc.) facilitan operaciones comunes como agregar, eliminar,

buscar y ordenar elementos. Como se demostró en los ejemplos, estas operaciones son esenciales para resolver problemas reales en áreas como análisis de datos, gestión de inventarios y procesamiento de información.

Uso del ciclo `for` en Python

Documento educativo

23 de agosto de 2026

Índice

1. Introducción	2
2. Sintaxis básica	2
3. Ejemplos básicos	2
3.1. Iterar sobre una lista	2
3.2. Iterar sobre una cadena de texto	2
3.3. Usar <code>range()</code>	2
4. Ciclo <code>for</code> con diccionarios	3
5. Uso de <code>break</code> y <code>continue</code>	3
6. Ciclo <code>for</code> con <code>else</code>	3
7. Ciclos <code>for</code> anidados	3
8. Comprensión de listas con <code>for</code>	3
9. Buenas prácticas	4
10. Conclusión	4

1. Introducción

El ciclo `for` en Python es una estructura de control que permite iterar sobre secuencias de datos como listas, tuplas, cadenas de texto, diccionarios y otros objetos iterables. Es una herramienta fundamental para la programación, ya que facilita la ejecución repetitiva de bloques de código.

2. Sintaxis básica

La sintaxis general del ciclo `for` en Python es la siguiente:

```
1 for variable in iterable:
2     # Bloque de código a ejecutar
3     pass
```

Listing 1: Sintaxis básica del ciclo `for`

Donde:

- `variable`: toma el valor de cada elemento del `iterable` en cada iteración.
- `iterable`: puede ser una lista, tupla, cadena, rango, etc.

3. Ejemplos básicos

3.1. Iterar sobre una lista

```
1 numeros = [1, 2, 3, 4, 5]
2 for num in numeros:
3     print(num)
```

Listing 2: Iterar sobre una lista de números

3.2. Iterar sobre una cadena de texto

```
1 palabra = "Python"
2 for letra in palabra:
3     print(letra)
```

Listing 3: Iterar caracteres de una cadena

3.3. Usar `range()`

La función `range()` es muy útil para generar secuencias numéricas.

```
1 for i in range(5):
2     print(i) # Imprime 0, 1, 2, 3, 4
```

Listing 4: Uso de `range()` con `for`

```
1 for i in range(2, 10, 2):
2     print(i) # Imprime 2, 4, 6, 8
```

Listing 5: `range()` con inicio

4. Ciclo for con diccionarios

```
1 estudiante = {"nombre": "Ana", "edad": 22, "carrera": "Ingenier a"}
2 for clave, valor in estudiante.items():
3     print(f"{clave}: {valor}")
```

Listing 6: Iterar sobre llaves y valores

5. Uso de break y continue

- break: termina el ciclo prematuramente.
- continue: salta a la siguiente iteración.

```
1 for i in range(10):
2     if i == 5:
3         break
4     print(i)    # Imprime 0,1,2,3,4
5
6 for i in range(10):
7     if i % 2 == 0:
8         continue
9     print(i)    # Imprime 1,3,5,7,9
```

Listing 7: Ejemplo con break y continue

6. Ciclo for con else

El bloque else se ejecuta si el ciclo no es interrumpido por un break.

```
1 for i in range(3):
2     print(i)
3 else:
4     print("Ciclo completado sin interrupciones")
```

Listing 8: Uso de else en for

7. Ciclos for anidados

```
1 for i in range(3):
2     for j in range(3):
3         print(f"({i},{j})", end=" ")
4     print()
```

Listing 9: For anidado para multiplicar matrices

8. Comprensión de listas con for

Es una forma concisa de crear listas.

```
1 cuadrados = [x**2 for x in range(10)]
2 pares = [x for x in range(20) if x % 2 == 0]
```

Listing 10: Comprensión de listas

9. Buenas prácticas

- Usar nombres descriptivos para las variables del ciclo.
- Evitar modificar la lista mientras se itera sobre ella.
- Preferir comprensión de listas para operaciones simples.
- Usar `enumerate()` cuando se necesite el índice.

```
1 colores = ["rojo", "verde", "azul"]  
2 for indice, color in enumerate(colores):  
3     print(f"{indice}: {color}")
```

Listing 11: Uso de `enumerate()`

10. Conclusión

El ciclo `for` es una de las estructuras más versátiles y utilizadas en Python. Permite recorrer cualquier tipo de datos iterable de manera sencilla y eficiente. Dominar su uso es esencial para cualquier programador que desee escribir código claro, legible y eficiente.